

Java Phone Digit Combinations

Generating letter combinations from a digit string

Objective / Aim

To write a Java program that generates all possible letter combinations represented by a string of phone keypad digits. The program builds each new set of combinations from the letters mapped to the next digit.

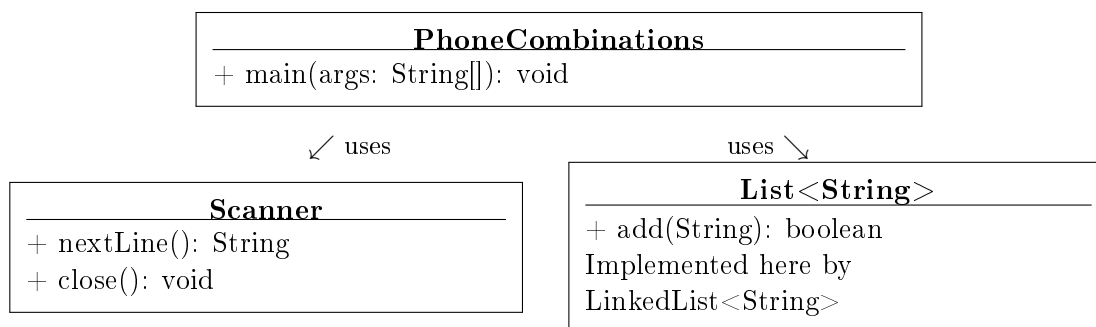
System Requirements

- **Hardware:** Standard computer with a Core i3/i5-class processor and 4 GB or more RAM.
- **Software:** Windows, Linux, or macOS; JDK 17 or JDK 21; and a text editor or Java IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

Algorithm / Logic Description

1. Read a string of digits from the user.
2. Store the phone keypad letter mapping in a string array. Digits 0 and 1 map to empty strings; digits 2 through 9 map to their corresponding letters.
3. Initialize a linked list with one empty string, which is the starting partial combination.
4. For each input digit, find its mapped letters and append each mapped letter to every combination generated so far.
5. Replace the current list with the newly generated list and continue until every input digit has been processed.
6. Print the final list of combinations.

UML Class Diagram



The diagram shows the program's dependencies: it uses **Scanner** for input and a **List<String>** (implemented by **LinkedList<String>**) to store combinations. The listed method is declared on the program class; temporary variables are omitted from the class attributes.

Source Code

Save the following program as `PhoneCombinations.java`:

```

import java.util.*;

public class PhoneCombinations {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter digits: ");
        String digits = sc.nextLine();

        String[] letters = {
            "", "", "abc", "def", "ghi",
            "jkl", "mno", "pqrs", "tuv", "wxyz"
        };

        List<String> combinations = new LinkedList<>();

        combinations.add("");

        for (char digit : digits.toCharArray()) {

            int number = digit - '0';

            List<String> newList = new LinkedList<>();

            for (String old : combinations) {

                for (char ch : letters[number].toCharArray()) {
                    newList.add(old + ch);
                }

            }

            combinations = newList;

        }

        System.out.println("Possible combinations:");

        System.out.println(combinations);

        sc.close();
    }
}

```

Compilation & Execution Commands

Open a terminal in the folder containing PhoneCombinations.java, then run:

```

javac PhoneCombinations.java
java PhoneCombinations

```

Output Screen

For the sample input 23, the program prints:

Enter digits: 23

Possible combinations:

[ad, ae, af, bd, be, bf, cd, ce, cf]

The output depends on the digits entered. For example, each digit 2 maps to **abc** and each digit 3 maps to **def**, so the input 23 produces nine combinations.